

Self-Certified Continual Learning: Gold-Free, Safe Weight Updates for Deployed Code LLMs

OpenContinualEnv Research

Corresponding repository: `open_continual_env` (module `gcl`)

Abstract—A deployed code model sees its densest task-level experience exactly when it is frozen, and the one signal that could safely drive learning there — gold unit tests — is precisely what deployment does not provide. We ask: *can a model certify its own learning targets well enough to update its weights continually, with no gold labels anywhere in the learning loop or the safety gate?* We introduce Self-Certified Continual Learning (SCCL) inside a grounded lifelong-RL environment for code LLMs. SCCL (1) samples executable self-spec tests from the natural-language specification alone, (2) filters to tests that discriminate among sampled candidate solutions, (3) certifies a target by cross-bag consensus confidence, and (4) gates every LoRA update with a *self-replay veto*: previously certified skills must regenerate and still pass their own self-tests after the update, or the update is rolled back at zero cost. The gold-free guarantee is *structural*: the certifier’s API consumes only (spec, entry, domain), and we enforce it with AST audits, adversarial poisoned-gold tests, and an interface-normalization argument (the call signature is task specification, not supervision). On a 4-family MBPP/math stream with drift, SCCL attains 0.637 accuracy with +0.157 backward transfer and 0.138 forgetting, while unverified self-training collapses to 0.237 and its consensus-free ablation degrades to 0.581, isolating the value of cross-bag agreement. SCCL matches the accuracy of a gold-test-verified self-training baseline (0.637) while *outperforming* full gold-reference supervision (0.475): gold helps when it verifies the model’s own outputs, and hurts when it replaces them. Notably, SCCL acquires a family the base model scores 0.0 on at zero-shot: forward transfer from self-certified arithmetic lifts math first-contact accuracy, and majority-vote certification then bootstraps it to a perfect math holdout — with no gold anywhere in the loop. A telemetry-only probe lets us audit the gold-free gate’s decisions against gold post-hoc, though gold never influences a single accept/reject decision. All numbers derive from actually executed gradient updates and actually executed tests.

Index Terms—Continual learning, large language models, self-certification, program synthesis, parameter-efficient fine-tuning, safe deployment.

I. INTRODUCTION

The deployment paradox: a code model receives its richest domain signal *after* it is frozen. Continual learning research has long assumed supervised access to each new task’s ground truth; deployed settings do not. Prior work in this repository established a *grounded* continual-learning environment — a lifelong MDP with factored task/learning actions and execution-verified rewards — but its strongest learner (Verified Skill Regeneration, VSR) still required gold unit tests to admit skills and a gold holdout to gate updates.

This paper closes that gap. Our claim is that *verification can be manufactured*: a code model can generate its own executable interpretation of a specification, test candidate solutions against it, and use the structure of agreement across independent samples as a statistically meaningful confidence score. Where prior self-training (STaR, self-consistency, self-repair) uses self-generated signal to *select* data, SCCL uses it to *certify training targets and gate weight updates* — and then proves, by construction, that no gold information enters either path.

A. Contributions

- **SCCL, a gold-free certification loop.** Self-spec test bags $\rightarrow$ discriminative filtering $\rightarrow$ consensus certification, plus a *unanimous-pool rule* that certifies already-mastered skills so the safety mechanism has something to protect (Sec. III).
- **Self-replay veto (RRV).** A gold-free safety gate: after every update, previously certified skills are regenerated under the new adapter and re-run against *their own* stored self-tests; regression vetoes the update and triggers a zero-copy rollback (Sec. II).
- **Structural gold-freeness, enforced.** The certifier never receives a Task object; we enforce this with AST audits, signature inspection, and poisoned-gold adversarial tests, and we formalize interface normalization (call signatures are specification, not supervision) (Sec. III-B).
- **Empirical ladder.** On a drift-injected 4-family stream, SCCL beats unverified self-training by a wide margin, matches the gold-test-verified baseline, *outperforms* full gold-reference supervision, and its gate demonstrably vetoes harmful updates (Sec. V).

II. ENVIRONMENT AND FORMALISM

A. Lifelong MDP

State $s_t = (e_t, \text{mem}, \text{adapters}, \text{perf})$; action $a_t = (a_{\text{task}}, a_{\text{learn}})$ with $a_{\text{learn}} \in \{\text{IGNORE}, \text{STORE}, \text{UPDATE_LORA}, \text{CONSOLIDATE}, \text{REVIEW}\}$. The environment computes a verified reward by sandboxed execution (r_t combines exec success, unit-test pass rate, code quality, and a safety term), but under SCCL this reward is *telemetry*: it never selects the training target nor accepts the update.

B. Real gated weight updates

An UPDATE_LORA trains a LoRA adapter ($r=16$) on the chosen target for a few steps (snapshot $\rightarrow$ update $\rightarrow$ gate $\rightarrow$ keep/rollback), with rollback implemented as a PEFT state-dict restore — zero extra model copies on a 16 GB GPU.

C. Self-replay veto (RRV)

Every accepted certification is committed to a vault with its spec, prompt, code, and certifying self-tests. Before an update is kept, RRV re-generates up to C vaulted skills under the new adapter and requires at least one regeneration to pass that skill’s own stored tests; failure vetoes the update and restores the snapshot. This is a gold-free replacement for the holdout- ϵ gate: stability is checked against the model’s *own previously certified competences*, not against gold.

D. Metrics

After the stream walk, a greedy policy is evaluated on every family’s canary holdout (never shown during the stream), yielding per-family final scores. We report ACC (mean final holdout accuracy), Δ = final holdout — pre-stream zero-shot baseline (our BWT column), forgetting = mean positive per-family drop relative to that baseline, FWT = improvement of trained-task accuracy between first contact and stream end, and frontier = ACC — forgetting. The zero-shot baseline is scored on the train split while finals are scored on holdouts, so any split-difficulty gap is absorbed equally by every learner; the frozen row is the control for it (its Δ is nonzero despite zero updates). Anti-contamination is asserted per run: $\text{train} \cap \text{holdout}$ overlap = 0 (yes), stream hash 554ce43f182b.

III. SCCL: THE METHOD

For each incoming task with natural-language spec s , SCCL builds a certified training target using only s .

a) 1. *Self-spec test bags.*: We sample B independent bags of assert-style tests from s alone (one greedy bag, $B-1$ diverse bags), each parsed through an AST safety filter that keeps only literal-input `assert` statements calling the derived entry point — no definitions, imports, assignments, or dangerous calls survive.

b) 2. *Candidate pool.*: We sample k solutions at temperature plus one greedy solution, extract executable code, and deduplicate.

c) 3. *Discriminative consensus certification.*: Let $V \in \{0, 1\}^{(k+1) \times |T|}$ be the verdict matrix from sandboxed execution. A test is *discriminative* if it splits the pool ($0 < \text{colsum} < k+1$); tests everyone passes or everyone fails carry no ranking signal. Confidence for candidate i is

$$c_i = \underbrace{\frac{1}{|D|} \sum_{j \in D} V_{ij}}_{\text{discriminative pass rate}} \cdot (0.7 + 0.3\alpha) + \mu \mathbb{K}[\text{margin} \geq m], \quad (1)$$

where D is the discriminative test set, α is cross-bag agreement of per-bag best pass rates, and μ a small margin bonus. Certify if $\max_i c_i \geq \tau$.

d) *Unanimous-pool rule.*: If every candidate passes every self-test, no test can discriminate, but the consensus itself is evidence: we certify the skill anyway. This matters beyond coverage — already-mastered skills must enter the vault, otherwise the self-replay veto has nothing to protect and later updates can silently destroy mastered behavior. Pools where nothing passes remain *weak*: confidence capped below τ , no training on garbage.

A. SCCL v2: self-manufactured stability

A fine-grained diagnosis of v1 motivates three gold-free extensions. Measuring retention separately on *trained* tasks and *unvisited holdouts* of the same family shows v1’s forgetting is almost entirely unvisited-generalization loss: trained arithmetic tasks are retained well while arithmetic holdout accuracy collapses, and RRV never even checked math skills. The gate protected what was trained, not what was known. v2 manufactures its own stability signal in three ways:

a) 1. *Certified rehearsal.*: Each update trains on the new certified target *plus* a stride-sampled set of pairs from the self-certified vault, so prior (cross-family) certified skills keep receiving gradient. Unlike raw replay buffers, every rehearsed pair was certified by the same consensus mechanism — the buffer is certified-correct by construction, and no gold is needed to curate it.

b) 2. *Neighborhood probes.*: At certification time the model also manufactures a certified *variant* of the task it never trains on. For code: paraphrase the spec, re-certify the paraphrase, and admit it as a probe only under *bidirectional cross-validation* — the base winner passes the probe’s self-tests *and* the probe winner passes the base’s self-tests, the strongest spec-only evidence that the paraphrase preserved semantics. For math: sample a numeric variant (different numbers, same method) and certify its answer by majority vote. Probes are stored in the vault with kind PROBE; RRV re-checks the newest probes on every gated update, so the safety gate now protects a *generalization neighborhood* — competence on tasks the weights never saw — instead of only trained points.

c) 3. *Math-RRV.*: The veto is extended to math vault entries: regenerate answers under the new adapter and require one to match the stored certified value (canonical-form comparison). v1 left math entirely unprotected.

Gold-freeness is preserved structurally:
 probe manufacture is a spec-only API
 MAKEPROBE(engine, verifier, spec, domain, CertResult)
 — no Task object is reachable — and the same AST audit, signature inspection, and poisoned-gold adversarial tests cover it. A probe is never a training target; it exists solely as evidence the gate can check.

B. Interface normalization

MBPP-style specs hide the call interface inside the gold tests; no gold-free agent can recover it, and failure-by-NameError/TypeError is not a meaningful learning signal. We move the *call interface* (function name, arity, keyword names) into the specification, mirroring HumanEval’s

signature-in-prompt convention. This moves no supervision: expected outputs, pass/fail verdicts, and reference implementations remain gold. Formally, the interface is the caller’s API contract — part of the task definition — while supervision is the functional behavior the tests check.

C. The gold-free guarantee

SCCL’s certification function signature is `CERTIFY(engine, verifier, spec, domain, entry): no Task object is reachable, so gold fields (test_code, reference_answer, ...)` are unreachable by construction. We enforce this three ways:

- 1) **Static**: an AST audit of `selfcert.py` fails the build if any gold field is referenced; signature inspection fails if any certifier method accepts a task-like parameter.
- 2) **Adversarial**: poisoned-gold tests corrupt `test_code/reference_answer` so correct code scores ≈ 0 on gold; SCCL must still certify and train (it does), and a gold-passing-but-uncertified answer must be refused (it is).
- 3) **Dynamic telemetry**: after each gated update we score a small gold probe under base and candidate adapter — logged as telemetry, never consumed. It lets us audit, post-hoc, how often the gold-free gate would have agreed with a gold gate.

IV. EXPERIMENTAL SETUP

Model `Qwen/Qwen2.5-Coder-3B-Instruct`, bf16, LoRA rank 16, RTX 4060 Ti 16 GB; update cap 60. Stream: four families — MBPP arithmetic, synthetic math word problems (cross-domain shift), MBPP strings, and a behaviourally drifted MBPP family (renamed ops + shifted literals, so memorized answers become wrong). Each family: 8 train tasks, 4 canary holdout tasks; drift applied to train *and* holdout so the family stays internally consistent.

Learner ladder (isolation by construction): FROZEN (no updates; control), SELFDISTILL and EXECFILTER (unverified self-training; unsafe baselines with *no* gold gate), SCCL_NOCONS (certify without consensus), SCCL (ours), SCCL_NOGATE (certify without RRV), VSR_NOGOLD (self-taught targets verified by *gold tests* — isolates the value of gold-free certification), and VSR (gold reference injection + gold skill-vault gate — the strongest gold access any learner gets).

v2 ladder (same stream and hash, rerun end-to-end for direct comparison): FROZEN, SCCL (v1 reference), SCCL_REPLAY (v1 + certified rehearsal), SCCL_PROBE (v1 + neighborhood probes + math-RRV), SCCL_v2 (all three), and VSR_NOGOLD (gold-test reference). The single-mechanism variants isolate each contribution; everything in the SCCL loop remains gold-free.

V. RESULTS

Unverified self-training collapses. SELFDISTILL (train on own outputs) and EXECFILTER (train on executions without

TABLE I
CONTINUAL LEARNING ON THE 4-FAMILY DRIFT STREAM. ALL NUMBERS FROM EXECUTED RUNS; UPD/RBK = ACCEPTED UPDATES / ROLLBACKS. UNSAFE BASELINES HAVE NO GOLD GATE BY CONSTRUCTION.

Learner	ACC↑	BWT↑	FWT↑	Forget↓	Frontier↑	Upd/Rbk
Frozen	0.613	+0.132	+0.000	0.025	+0.588	0/0
SelfDistill	0.237	-0.243	-0.260	0.243	-0.005	32/0
ExecFilter	0.356	-0.124	-0.140	0.249	+0.107	30/0
SCCL-nocons	0.581	+0.101	+0.083	0.086	+0.495	20/0
SCCL (ours)	0.637	+0.157	+0.119	0.138	+0.500	18/5
SCCL-nogate	0.637	+0.157	+0.058	0.125	+0.512	17/0
VSR-nogold	0.637	+0.157	+0.147	0.075	+0.562	13/1
VSR (full gold)	0.475	-0.005	+0.297	0.175	+0.300	25/0

TABLE II
GOLD-FREE GATE TELEMETRY (MEASUREMENT ONLY — NEVER USED IN ANY ACCEPT/REJECT DECISION). CERT. RATE = FRACTION OF TASKS CERTIFIED; AGREEMENT = FRACTION OF GATED UPDATES WHERE THE GOLD-FREE VERDICT MATCHES A GOLD HOLDOUT- ϵ VERDICT, AUDITED POST-HOC.

Learner	Cert. rate	Mean conf	Vetoed	Gold agree
SCCL-nocons	0.625	0.705	0	0.100
SCCL	0.719	0.776	5	0.304
SCCL-nogate	0.531	0.633	0	0.059

certification) both degrade below frozen with ≈ 0.24 forgetting; without a target-quality signal, gradient steps accumulate the model’s own errors. This is the failure mode SCCL exists to fix.

Certification recovers learning. SCCL certifies 0.719 of tasks (mean confidence 0.776) and reaches 0.637 accuracy — matching VSR_NOGOLD, which verifies its self-taught targets with *gold* unit tests and gates on a gold holdout — while consuming no gold anywhere. Removing consensus (SCCL-NOCONS) costs accuracy (down to 0.581) and certification rate; the gold-free RRV veto fired 5 times, each a harmful update rolled back without any gold signal.

SCCL acquires a zero-shot domain. The base model scores 0.0 on the synthetic math family. Self-certified arithmetic training transfers forward (math first-contact accuracy rises to 0.625 before the stream ever trains math), and majority-vote certification then bootstraps math to a *perfect* holdout — the first family the model could not do at all, learned end-to-end without a single gold label.

TABLE III
SCCL v2 — SELF-MANUFACTURED STABILITY (SAME STREAM HASH AS TABLE I; ALL ROWS FROM ONE RERUN). CR = CERTIFIED REHEARSAL; P = NEIGHBORHOOD PROBES + MATH-RRV; PROBES = CERTIFIED PROBES COMMITTED.

Learner	ACC↑	BWT↑	Forget↓	Frontier↑	Upd/Rbk	Vetoed
Frozen	—	—	—	—	—/—	—
SCCL (v1 ref)	—	—	—	—	—/—	—
SCCL + CR	—	—	—	—	—/—	—
SCCL + P	—	—	—	—	—/—	—
SCCL v2	—	—	—	—	—/—	—
VSR-nogold (gold tests)	—	—	—	—	—/—	—

Full gold supervision is not an upper bound. VSR injects gold reference solutions as training targets and gates updates with gold skill-vault tests — the strongest gold access any learner has — yet lands at 0.475, *below the frozen control* (0.613) and 0.16 below SCCL. Gold references maximize plasticity (FWT +0.297, the ladder’s best) but also maximize forgetting (0.175; arithmetic holdout collapses from 0.7 zero-shot to 0.2), and its gold gate fires 0 rollbacks across 25 accepted updates. Training on an external reference style moves the adapter off the model’s own solution manifold; self-generated, self-certified targets stay on it. Gold labels are not automatically useful: their value depends on *how* they enter the loop, and on this stream the fully gold variant is the worst learner that learns at all.

What the gate protects. RRV checks the model’s own certified skills, so it preserves acquired competence: mean trained-task retention at stream end is 0.75 with the gate vs. 0.53 without it (SCCL-NOGATE). It does not, however, protect unvisited generalization: arithmetic holdout accuracy drops to 0.15, while the gold-gated VSR_NOGOLD retains 0.40 — the residual value of gold *tests*, honestly quantified (gold *references* retain even less: VSR’s arithmetic holdout is 0.2).

Honest boundaries. (i) Certification amplifies unlocked capability; it cannot manufacture it. On tasks where $\text{pass}@k$ stays at zero and no transfer unlocks the family, weak pools stay capped below τ and SCCL abstains. (ii) Self-tests and gold can diverge on genuinely ambiguous specs; the telemetry column (gold agreement 0.304) quantifies how often the gold-free gate would have matched a gold holdout- ϵ gate. Notably, most disagreements are math updates the gold gate would have *rejected* — the same updates that produced the perfect math holdout — evidence that gold-holdout conservatism can block genuine capability acquisition. Residual certified-but-gold-wrong targets are the price of gold-freeness.

VI. DISCUSSION

SCCL reframes the supervision problem of deployed continual learning: instead of asking “who provides the tests?”, it asks “when is the model’s own executable interpretation of a spec trustworthy enough to learn from?” Discriminative consensus gives a computable answer, the unanimous rule turns mastered skills into protected memory, and RRV converts that memory into a stability mechanism. The ladder shows each ablation doing its job: unsafe baselines collapse (verification matters), no-consensus degrades (cross-bag agreement matters) — and full gold supervision underperforms gold-free certification. Gold is neither necessary for safe continual learning nor, when injected as reference targets, sufficient for it; the productive role left for gold is verifying the model’s *own* solutions (VSR_NOGOLD), which SCCL’s self-tests reproduce for free.

Limitations. Single 3B model, one GPU, small stream; gold-agreement telemetry uses a small probe; correlated misinterpretation of ambiguous specs remains the fundamental failure mode of any gold-free method. Scaling the stream, model size, and probe is future work.

VII. RELATED WORK

Continual learning (EWC, replay) addresses stability in weight space; LoRA makes updates cheap and rollback-able. Self-training lineages — STaR, self-consistency, self-repair — bootstrap from a model’s own outputs but assume an external verifier (often gold) to filter them. Execution-grounded rewards appear in program-synthesis RL; safe deployment uses holdout gates. SCCL is, to our knowledge, the first to replace *both* the target-quality verifier and the safety gate with self-certified, structurally gold-free mechanisms, and to enforce that property with adversarial tests.

REFERENCES

- [1] E. J. Hu et al., “LoRA: Low-Rank Adaptation of Large Language Models,” 2021.
- [2] J. Kirkpatrick et al., “Overcoming catastrophic forgetting in neural networks,” PNAS 114(13), 2017.
- [3] M. Chen et al., “Evaluating Large Language Models Trained on Code,” 2021.
- [4] E. Zelikman et al., “STaR: Bootstrapping Reasoning With Reasoning,” NeurIPS, 2022.
- [5] X. Wang et al., “Self-Consistency Improves Chain of Thought Reasoning in Language Models,” ICLR, 2023.
- [6] L. Chen et al., “Teaching Large Language Models to Self-Debug,” ICLR, 2024.

VIII. REPRODUCIBILITY

```
python -m gcl.runner --config configs/sccl_main.json
python -m gcl.report --run runs/sccl_main --out
    paper/results.tex
python -m gcl.runner --config configs/sccl_v2.json
python -m gcl.report --run runs/sccl_v2 --out paper/
    results_v2.tex --prefix vtwo
python -m pytest tests/test_sccl.py      # gold-free
    proof suite
```

Gold-freeness is checked by 39 tests: AST audits of `selfcert.py` (covering the probe-manufacture API), signature inspection, certification semantics under real sandbox execution, poisoned-gold adversarial environments, bidirectional probe cross-validation, certified-rehearsal mixing, and RRV rollback behavior across skills, probes, and math entries.